

SAMS / MarkIn Beginner Technical Guide

A complete, beginner-friendly explanation of the Smart Attendance Management System

Generated on 2026-05-20

Purpose of this PDF

This guide is written for a beginner who knows little or nothing about the project. It explains what each part does, why it exists, how data moves, how attendance is calculated, how AI face recognition works, and how to debug the system without exposing secrets.

The project is a full-stack attendance management system. It has a React frontend, an Express backend, a MongoDB database, and a Python FastAPI AI service. Teachers create classes, students join them, attendance can be taken manually, through QR code, or from classroom photos, and both students and teachers see analytics. Email OTP and notification flows make the system closer to a real project.

This PDF intentionally explains basic ideas before technical details. If you already know web development, you can skim the beginner boxes. If you are new, read slowly from the start: the same concepts repeat in different features, so the project will become clearer chapter by chapter.

Important safety note:

- This guide never prints real secrets from .env files.
- Use placeholder values such as EMAIL_PASSWORD=<gmail-app-password>.
- Do not paste real database URLs, app passwords, or OTPs in reports.

Contents Overview

This is a study map for the guide. Page numbers are intentionally omitted because the editable source and generated PDF can change when new sections are added.

- How to read this guide and the big picture of SAMS / MarkIn.
- Architecture, service boundaries, technology stack, and folder tour.
- Frontend routing, React patterns, beginner web concepts, and backend service structure.
- Database models, sample documents, and data lifecycles.
- Authentication, OTP, password reset, class management, attendance, AI, email, and exam eligibility.
- Student journey, teacher journey, admin controls, deployment, testing, troubleshooting, and security.
- API examples, complete day-in-the-life walkthrough, viva question bank, glossary, and code reading workbook.

Chapter 1: How To Read This Guide

A software project can look frightening because many files work together. The trick is to stop reading file by file and start reading flow by flow. A flow means a real action, such as login, joining a class, taking attendance, sending an OTP, or calculating exam eligibility. Once you understand the flow, the files become easier to remember.

The simplest mental model

Think of SAMS as four cooperating workers. The browser worker shows buttons and dashboards. The backend worker checks rules and protects the data. The database worker remembers everything. The AI worker looks at face images and returns suggestions. No single worker does everything.

```
User clicks button in browser
      |
      v
React frontend calls backend REST API
      |
      v
Express backend validates request and applies rules
      |
      +--> MongoDB stores or reads documents
      |
      +--> FastAPI AI service runs face recognition when needed
      |
      +--> SMTP email service sends OTPs and notifications
      |
      v
Frontend receives JSON and updates the dashboard
```

Recommended reading path

- First read Chapters 1 to 4 to understand the architecture and folder structure.
- Then read the authentication, class, and attendance chapters because they explain the core project.
- Read the AI chapter slowly. It explains face detection, face recognition, embeddings, thresholds, and alternatives.
- Use the appendices as quick references when you are looking at code.

What beginner words mean

A route is a URL handled by the backend. A controller is the function that receives the request for that route. A service is where the real work is done. A model is a database schema. A component is a reusable frontend UI function. A dashboard is not only design; it is a screen built from API data.

Chapter 2: Project Overview

SAMS, also called MarkIn in the product wording, is a Smart Attendance Management System. The word smart does not only mean AI. It means the system tries to make attendance useful after it is recorded. Students see percentages, weak classes, upcoming exams, recovery plans, and alerts. Teachers see rosters, attendance drafts, pending actions, class schedules, and students who need attention. Admins can clean incorrect classes and accounts.

Main user roles

Role	Main goal	Important screens	Important backend data
Student	Join classes, track attendance, enroll face, submit leave proof, watch exam eligibility.	Student dashboard, class detail, face enrollment, notifications, profile.	User, ClassMembership, AttendanceRecord, FaceProfile, LeaveRequest, ClassExam.
Teacher	Create classes, manage roster, take attendance, review leave, set exams, archive finished classes.	Teacher dashboard, class workspace, students page, sessions page, exam section.	Classroom, ClassMembership, AttendanceDraft, AttendanceRecord, LeaveRequest, ClassExam.
Admin	Control system records and remove wrong or old data.	Admin dashboard.	User, Classroom, linked data across attendance and class collections.

Main features in plain language

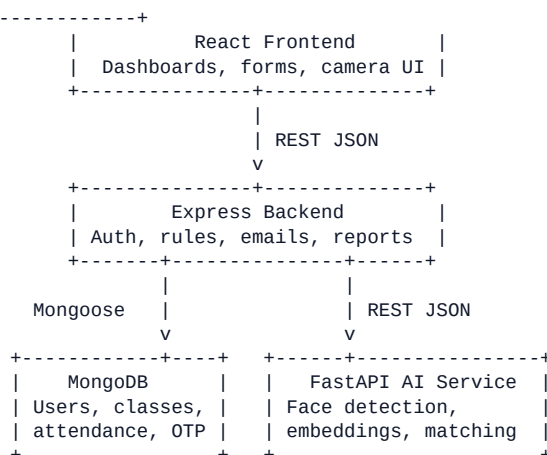
- Account system: students and teachers register, verify email through OTP, and then log in.
- Password reset: user requests OTP, waits one minute before resend, verifies OTP, then enters the new password twice.
- Class management: teachers create classes with subject details, room, schedule, and join code.
- Student joining: students join a teacher class using a join code.
- Face enrollment: students upload multiple face images so the AI service can build an identity profile.
- Attendance methods: teacher can mark manually, create a QR attendance session, or run AI on a classroom photo.
- Today Attendance Data: AI result becomes an editable draft before final attendance is saved.
- Absence email: as soon as draft attendance marks a student absent, the student gets an email so mistakes can be reported early.
- Exam eligibility: teacher sets exam date and required attendance percentage; students see whether they are eligible and how many future classes they must attend.
- Leave proof: students upload PDF/image documents for absence reason; teacher reviews and approves or rejects.
- Class archive: when a semester/class is over, teacher can end the class and save summary data.

Why this is not a single app file

A beginner often asks why the project is split into frontend, backend, database, and AI service. The reason is responsibility. The frontend should not store passwords or talk directly to AI model files. The backend should not perform heavy image recognition inside the same process that handles every dashboard request. MongoDB should store data, not business rules. The AI service should focus on model loading and inference. This separation makes the system easier to debug and safer to grow.

Chapter 3: Architecture And Service Boundaries

Architecture means the way the project is divided into pieces and how those pieces communicate. In SAMS, the pieces are intentionally separated. This makes it easier to replace one piece later. For example, if the AI service becomes slow, it can be moved to a stronger server without rewriting the React dashboard.



Frontend boundary

The frontend lives in `frontend/`. It is a React application. It should ask the backend for data and render the answer. It should not decide whether a password is correct, whether an OTP is valid, whether a student is in a class, or whether a face match is accepted. Those decisions belong to backend and AI service logic.

Backend boundary

The backend lives in `backend/`. It is an Express REST API. It is the source of truth for users, classes, attendance records, leave requests, exam rules, email notifications, and admin actions. It also acts as a gateway to the AI service. The frontend never directly calls the AI service in the current architecture.

AI service boundary

The AI service lives in `ai-service/`. It is written in Python using FastAPI. It loads InsightFace models, creates face embeddings, compares those embeddings, and returns recognition suggestions. It does not decide the final attendance record. The teacher and backend still control finalization.

Database boundary

MongoDB stores documents. The backend uses Mongoose models to define the shape of those documents. MongoDB does not know what a teacher dashboard is; it only stores collections such as users, classrooms, attendance records, and OTP records. The backend combines those documents into useful dashboard responses.

Why REST JSON

REST JSON is a simple communication style. The frontend sends an HTTP request to a URL, usually with JSON data. The backend returns JSON. This is beginner friendly because you can test the same URL with curl or Postman. Alternatives include GraphQL, gRPC, WebSockets, or server-rendered pages. REST is selected here because the workflows are request-response based and easy to debug.

Chapter 4: Technology Stack

Layer	Technology	Why it is used
Frontend	React 18, Parcel, JavaScript, CSS	Builds the browser UI and dashboards. Parcel keeps setup simple for a beginner project while React makes screens reusable.
Backend	Node.js, Express, Mongoose	Owns the REST API, authentication, business rules, MongoDB reads/writes, email, and AI integration.
Database	MongoDB	Stores flexible documents such as users, classrooms, attendance records, drafts, exams, leave requests, OTPs, and email logs.
AI service	Python, FastAPI, InsightFace, ONNX Runtime	Runs face enrollment, face detection, face recognition, and classroom attendance inference.
Email	Nodemailer with SMTP	Sends OTPs, welcome emails, leave updates, absence alerts, exam warnings, and class cancellation messages.
DevOps	Docker Compose	Runs frontend, backend, AI service, and MongoDB together during local or hosted deployment.

Alternatives and why this project uses the current choices

Area	Current choice	Alternatives	Why current choice is reasonable here
Frontend	React + Parcel	Vue, Angular, Next.js, plain HTML/JS	React is common, component-based, and fits dashboards. Parcel avoids a heavy config file, which is useful for a college project.
Backend	Express	NestJS, Fastify, Django, Spring Boot	Express is small and direct. Controllers, services, and routes are easy for beginners to trace.
Database	MongoDB	PostgreSQL, MySQL, SQLite, Firebase	Attendance documents have many related shapes: drafts, exams, attachments, discussion, logs. MongoDB is flexible and pairs naturally with Node/Mongoose.
AI API	FastAPI	Flask, Django REST, Node Python bridge	FastAPI gives type-friendly request schemas and automatic docs, and Python is the natural ecosystem for ML libraries.
Face recognition	InsightFace ArcFace	OpenCV LBPH, Dlib, FaceNet, DeepFace wrapper, custom CNN	ArcFace embeddings are strong for identity matching and do not require training a model from scratch for this project.
Email	SMTP through Nodemailer	SendGrid, Mailgun, SES, Brevo API	SMTP can work with free Gmail app passwords and keeps the project usable without paid services.

Beginner explanation: library vs framework

A library is code you call when you need it. A framework often calls your code according to its rules. React is often described as a UI library, while Express is a minimal web framework. In practice, both help avoid writing everything from zero. Mongoose is a library for MongoDB models, Nodemailer is a library for email, and InsightFace is a model toolkit for face analysis.

Why not keep everything inside one MERN server

The MERN stack normally means MongoDB, Express, React, and Node. SAMS adds a Python AI service because deep learning libraries and ONNX Runtime support are stronger in Python. Keeping AI separate prevents the Node backend from becoming large, slow, and hard to deploy. The backend becomes a coordinator rather than a model host.

Chapter 5: Project Folder Tour

Before understanding code, you should understand where to look. A beginner mistake is opening random files and getting lost. Use the folder names as a map.

SAMS/	
frontend/	React application shown in the browser
backend/	Express API, MongoDB models, services, email, auth
ai-service/	Python FastAPI service for face recognition
shared/	Shared API contract examples
docs/	Architecture notes and diagrams
reports/	Generated reports and PDF guides
docker-compose.yml	
package.json	

Frontend folders

Path	Meaning
frontend/src/App.jsx	Hash-based router that chooses which page component to show.
frontend/src/pages/AuthPage	Signup, login, reset password, OTP forms.
frontend/src/pages/StudentDashboardPage	Student dashboard, tools, class analytics, schedule, notifications.
frontend/src/pages/TeacherDashboardPage	Teacher dashboard, class creation, profile, exams, schedules.
frontend/src/pages/TeacherClassroomPage	Class workspace, camera/photo attendance, manual attendance, Today Attendance Data.
frontend/src/pages/AdminDashboardPage	Admin controls for users/classes.
frontend/src/services/api.js	All frontend REST calls to the backend.
frontend/src/services/session.js	Local browser session storage helpers.

Backend folders

Path	Meaning
backend/src/routes/index.js	Maps API URLs to controller functions.
backend/src/controllers	Thin request/response layer.
backend/src/services	Business logic: auth, dashboards, attendance, email, AI gateway.
backend/src/models	Mongoose schemas for MongoDB collections.
backend/src/config	Environment and database connection.
backend/src/app.js	Express app, CORS, JSON parser, router.
backend/src/server.js	Database connect, admin seed, email check, scheduler, listen.

AI service folders

Path	Meaning
ai-service/app/api/routes.py	FastAPI endpoints for models, enrollment, recognition, verification, finalization.
ai-service/app/pipelines	High-level AI workflows: enrollment, classroom attendance, live verification.
ai-service/app/services	Face detection and recognition model logic.
ai-service/app/storage	Local JSON stores for enrolled profiles and AI sessions.
ai-service/app/schemas	Pydantic request/response models.
ai-service/app/core/config.py	AI thresholds, model pack names, model directories, providers.

Chapter 6: Frontend Explained

The frontend is the part the user sees. It runs in the browser at the development URL, usually `http://localhost:5173`. It is built with React, which means each screen is made from components. A component is a JavaScript function that returns UI.

Routing

The project uses hash routing. A URL like `http://localhost:5173/#/student-dashboard` tells `App.jsx` to render `StudentDashboardPage`. Hash routing is simple because the development server can serve the same `index.html` for every route. The browser handles the part after `#`.

Route	Component	Purpose
/	HomePage	Landing page with feature explanation and service status.
/login	AuthPage login	Student, teacher, and admin login.
/signup	AuthPage signup	Student/teacher registration with email OTP.
/reset-password	AuthPage reset	OTP reset with resend cooldown and password confirmation.
/student-dashboard	StudentDashboardPage	Main student overview with metrics and tools.
/student-classes	StudentDashboardToolsPage	Student class list and analytics tools.
/student-performance	StudentDashboardToolsPage	Attendance performance and recovery plan.
/student-schedule	StudentDashboardToolsPage	Student timetable view.
/student-exams	StudentDashboardToolsPage	Exam calendar and eligibility details.
/student-notifications	StudentDashboardToolsPage	Notification page including alerts.
/student-classroom	StudentClassDetailPage	Detailed page for one joined class.
/student-face-enrollment	StudentFaceEnrollmentPage	Face profile enrollment workflow.
/teacher-dashboard	TeacherDashboardPage	Teacher overview, classes, exams, schedules.
/teacher-classroom	TeacherClassroomPage	Class workspace.
/teacher-classroom-attendance	TeacherClassroomPage attendanceOnly	Focused attendance page.
/teacher-classroom-sessions	TeacherClassroomSessionsPage	Attendance session history.
/teacher-classroom-students	TeacherClassroomStudentsPage	Roster management.
/admin-dashboard	AdminDashboardPage	Admin record control panel.

Frontend data pattern

```

Component loads
  |
  v
useEffect calls a function from frontend/src/services/api.js
  |
  v
api.js uses fetch() to call backend
  |
  v
Backend returns JSON
  |
  v
Component saves JSON in useState
  |
  v
React re-renders the page with updated dashboard data
  
```

Why api.js matters

The frontend does not write fetch code everywhere. Most API calls are collected in `frontend/src/services/api.js`. This is good because if the backend base URL changes, or if error handling changes, the project can update one service file instead of every page.

Session storage

After login, the frontend stores the user session in browser storage through `frontend/src/services/session.js`. This is not the same as strong production authentication with secure HTTP-only cookies or JWT refresh tokens. For this project it is a simple way to remember who is logged in. The frontend checks the saved session and redirects if it becomes invalid.

Student dashboard design

The student dashboard is meant to answer practical questions: How many classes have I joined? Which class is weak? What is my attendance percentage? Do I need to attend more classes before an exam? What notifications need attention? This is why separate tool pages exist for classes, performance, schedule, exams, insights, and notifications. It avoids putting every tool on one crowded page.

Teacher dashboard design

The teacher dashboard is focused on action. A teacher creates classes, sees managed classes, sets exams, checks schedules, and opens class workspaces. The class workspace is where high-risk operations happen: attendance, draft correction, finalization, student roster changes, QR sessions, cancellation, leave review, and export.

Admin dashboard design

The admin dashboard is intentionally simpler than a teacher dashboard. It should not show teacher-only or student-only metrics such as leave requests or upcoming exams as primary cards. Admin features focus on deleting incorrect users/classes, toggling email verification, and checking overall record counts.

Chapter 7: Beginner Web Development Concepts

This chapter explains the web development ideas that appear again and again in the project. If you understand these, the code will feel much less mysterious.

Concept	Meaning
HTTP request	A message sent from browser or client to a server. It has a method, URL, headers, and sometimes a body.
HTTP response	The answer from the server. It has a status code and usually JSON data in this project.
GET	An HTTP method for reading data. Example: load student dashboard.
POST	An HTTP method for creating or triggering work. Example: signup or create class.
PATCH	An HTTP method for partial updates. Example: approve leave or update draft attendance.
DELETE	An HTTP method for deleting a record. Example: admin deletes a class.
JSON	A text format for structured data. Frontend and backend use it to talk.
Status code 200	Success. The request worked.
Status code 201	Created. A new record was successfully created.
Status code 400	Bad request. The user sent missing or invalid data.
Status code 401	Unauthorized. Login or credentials failed.
Status code 404	Not found. The requested record does not exist.
Status code 500	Server error. Something unexpected happened.
CORS	Browser rule that decides which frontend origins can call this backend.
React state	Data stored inside a component. When state changes, the UI re-renders.
useEffect	React hook used to run side effects such as fetching dashboard data after the page loads.
Component	A reusable piece of UI written as a JavaScript function.
Props	Values passed from a parent component into a child component.
Hash route	A frontend URL route after #, such as #/student-dashboard.
Mongoose schema	A JavaScript definition of fields, validation, defaults, and indexes for MongoDB.
Index	A database helper structure that makes lookups faster and can enforce uniqueness.
Lean query	A Mongoose query that returns plain objects instead of full model documents.
Data URL	A base64 encoded file stored as text, often used for small image or PDF attachments in this project.
Scheduler	A background process that periodically checks pending work, such as old attendance drafts.
Gateway	A service that forwards a request to another service. The backend acts as an AI gateway.

What actually happens when you click a button

A button in React is usually connected to an event handler such as `onClick` or `onSubmit`. The handler may update local state, validate form data, or call an API function. If it calls the backend, the browser sends an HTTP request. The backend route receives it, a controller calls a service, the service reads or writes MongoDB, and the response returns to the component.

```

React button
  -> event handler
  -> api.js function
  -> fetch()
  -> Express route
  -> controller
  -> service
  -> Mongoose model
  -> MongoDB
  -> JSON response

```

```
-> setState()  
-> updated UI
```

Why the frontend should not trust itself

The frontend can make the user experience nicer, but it cannot be the only security layer. A user can edit browser JavaScript, call APIs manually, or bypass disabled buttons. That is why important rules also exist on the backend. Example: OTP resend has a frontend timer, but backend also rejects early resend. Archive blocks are also checked on the backend.

Why JSON responses are shaped for dashboards

The backend often sends dashboard-ready objects instead of raw database documents. For example, the student dashboard response can include `classPerformance`, `comparison`, `upcomingClasses`, `alerts`, and `exams`. This keeps the React component focused on display instead of doing heavy calculations in the browser.

Frontend error messages

The API helper reads JSON error responses and throws JavaScript Error objects. Components catch these errors and show messages. This pattern makes errors visible to the user while keeping the fetch details inside `api.js`.

Chapter 8: Backend Explained

The backend is the project's rule keeper. If a student tries to join a class, the backend checks the join code and creates a `ClassMembership`. If a teacher finalizes attendance, the backend writes `AttendanceRecord` documents. If a password is reset, the backend checks OTP rules and hashes the new password. This is why the backend is the most important part of the system.

Backend startup

```
server.js startup sequence:
1. create Express app
2. connect to MongoDB
3. ensure default admin account exists
4. verify email service in background
5. start attendance draft scheduler
6. listen on configured backend port
```

Controller-service-model pattern

SAMS uses a common pattern. Routes decide which controller handles a URL. Controllers read request data and return JSON. Services contain the real business logic. Models talk to MongoDB. This separation helps the project stay readable.

```
HTTP Request
|
v
Route in backend/src/routes/index.js
|
v
Controller in backend/src/controllers/*.js
|
v
Service in backend/src/services/*.js
|
v
Mongoose Model in backend/src/models/*.js
|
v
MongoDB document collection
```

API endpoint summary

Method	Endpoint	Purpose
GET	/health	Backend, database, AI, and email readiness.
GET	/email/status	Safe email service status without credentials.
POST	/email/verify	Force SMTP verification.
POST	/email/test	Send a protected test email.
POST	/auth/signup	Create student or teacher account and send verification OTP.
POST	/auth/login	Login with role, user ID, and password.
POST	/auth/verify-email	Activate account after signup OTP.
POST	/auth/resend-verification	Resend signup OTP after cooldown.
POST	/auth/password-reset/request	Send password reset OTP.
POST	/auth/password-reset/verify	Verify reset OTP and issue reset session.
POST	/auth/password-reset/confirm	Set new password after reset session.
GET	/students/:userId/dashboard	Student dashboard data.
POST	/students/:studentId/classes/join	Join a class using join code.

Method	Endpoint	Purpose
POST	/students/:studentId/classes/:classId/leave-requests	Submit leave/medical proof.
GET	/students/:studentId/face-profile	Get face enrollment status.
POST	/students/:studentId/face-profile/enroll	Enroll student face images.
GET	/teachers/:teacherId/dashboard	Teacher dashboard data.
POST	/teachers/:teacherId/classes	Create a class.
PATCH	/teachers/:teacherId/classes/:classId/exam	Set exam date and attendance rule.
GET	/teachers/:teacherId/classes/:classId	Load teacher class workspace.
PATCH	/teachers/:teacherId/classes/:classId/archive	End/archive a class and save summary.
POST	/teachers/:teacherId/classes/:classId/students	Add a student to teacher class roster.
PATCH	/teachers/:teacherId/classes/:classId/students/:studentId	Update a roster student.
DELETE	/teachers/:teacherId/classes/:classId/students/:studentId	Remove a roster student.
POST	/teachers/:teacherId/classes/:classId/attendance/manual	Submit manual attendance.
POST	/teachers/:teacherId/classes/:classId/cancel-today	Cancel today's class and notify students.
POST	/teachers/:teacherId/classes/:classId/attendance/qr-session	Create QR attendance token.
POST	/students/attendance/qr-scan	Student marks attendance through QR scan.
POST	/teachers/:teacherId/classes/:classId/attendance/session	Run AI classroom photo attendance.
PATCH	/teachers/:teacherId/classes/:classId/attendance/today-draft/:draftId	Edit Today Attendance Data.
POST	/teachers/:teacherId/classes/:classId/attendance/today-draft/:draftId/finalize	Finalize draft attendance.
PATCH	/teachers/:teacherId/classes/:classId/leave-requests/:requestId	Approve or reject leave request.
GET	/admins/:adminId/dashboard	Admin dashboard.
DELETE	/admins/:adminId/classes/:classId	Delete class and related data.
PATCH	/admins/:adminId/classes/:classId/status	Activate/archive a class.
DELETE	/admins/:adminId/users/:role/:userId	Delete student or teacher and related data.

Why services are useful

A beginner might ask why controller code cannot directly create database documents. It can, but then controllers become huge. For example, teacher attendance has to validate the teacher, validate class status, call AI, build a draft, send absent emails, and return a clean response. That logic belongs in a service because it is more than just reading a request and returning a response.

Important backend services

Service file	Main responsibility
authService.js	Registration, login, admin seed, email verification OTP, password reset OTP/session.
classroomService.js	Create classes, join classes, list teacher/student classes.
teacherClassroomService.js	Teacher class workspace, roster, attendance, drafts, QR, cancellation, archive.
studentDashboardService.js	Combines memberships, attendance, exams, leave, face profile into student dashboard data.
teacherDashboardService.js	Combines managed classes, rosters, attendance summaries, exams into teacher dashboard data.
attendanceAnalyticsService.js	Calculates class averages, student percentages, sessions, flagged students.

Service file	Main responsibility
examScheduleService.js	Exam date rules and eligibility math.
emailService.js	SMTP transporter, queue, retries, delivery logs, service status.
emailNotificationService.js	Builds and sends notification events using email templates.
aiService.js	Backend gateway for AI service calls and timeout/retry handling.

Chapter 9: Backend Services Deep Dive

Services are the muscles of the backend. Controllers are small because services do the hard work. This chapter explains why each important service exists.

authService.js

Main responsibility: Owns account creation, login, email verification, password reset, default admin seed, and user sanitization.

This service protects identity. It normalizes user IDs, validates signup data, hashes passwords, creates OTPs, and prevents login before verification.

- Find the exported function names in this file.
- Find which controller calls those functions.
- Find which models are imported at the top.
- Find the validation errors thrown by the service.

classroomService.js

Main responsibility: Creates classes, generates join codes, joins students, and lists classes for dashboards.

This service connects users to classrooms. It is the start of nearly every attendance feature.

- Find the exported function names in this file.
- Find which controller calls those functions.
- Find which models are imported at the top.
- Find the validation errors thrown by the service.

teacherClassroomService.js

Main responsibility: Large service for the teacher class workspace.

It handles roster updates, manual attendance, QR, AI attendance sessions, drafts, finalization, cancellation, leave review, exam setting, and archive guards.

This is one of the largest services because a class workspace has many actions. When reading it, do not try to memorize everything. Read one flow at a time: get classroom, add student, manual attendance, QR attendance, AI session, draft update, draft finalize, leave review, exam setting, archive.

- Find the exported function names in this file.
- Find which controller calls those functions.
- Find which models are imported at the top.
- Find the validation errors thrown by the service.

studentDashboardService.js

Main responsibility: Builds the rich student dashboard response.

It joins class memberships, attendance records, leave requests, face profile, exams, schedule, alerts, and recovery calculations.

This service is a good example of dashboard aggregation. It does not merely return User plus ClassMembership. It calculates useful derived data: attendance trend, class comparison, recovery plan, AI coach notes, weekly schedule, upcoming classes, cancellation alerts, low attendance alerts, and exam alerts.

- Find the exported function names in this file.
- Find which controller calls those functions.
- Find which models are imported at the top.
- Find the validation errors thrown by the service.

teacherDashboardService.js

Main responsibility: Builds the teacher dashboard response.

It calculates classes managed, flagged students, schedules, upcoming exams, attendance averages, and archived class information.

- Find the exported function names in this file.
- Find which controller calls those functions.
- Find which models are imported at the top.
- Find the validation errors thrown by the service.

attendanceAnalyticsService.js

Main responsibility: Converts raw attendance records into percentages and sessions.

It decides how present, late, absent, and cancelled records affect totals and dashboard metrics.

- Find the exported function names in this file.
- Find which controller calls those functions.
- Find which models are imported at the top.
- Find the validation errors thrown by the service.

examScheduleService.js

Main responsibility: Owns exam rules and eligibility calculations.

It counts remaining scheduled classes and calculates whether each student can reach the required percentage.

This service shows why business logic belongs outside the component. Exam eligibility involves dates, schedules, percentages, and edge cases. Keeping it in the backend makes student and teacher dashboards consistent.

- Find the exported function names in this file.
- Find which controller calls those functions.
- Find which models are imported at the top.
- Find the validation errors thrown by the service.

emailService.js

Main responsibility: Low-level email delivery engine.

It creates the SMTP transporter, verifies connection, queues sends, retries failures, and stores delivery logs.

This service is intentionally lower level than emailNotificationService. It should not know what a leave request means. It only knows how to send mail, retry, queue, and log results.

- Find the exported function names in this file.
- Find which controller calls those functions.
- Find which models are imported at the top.
- Find the validation errors thrown by the service.

emailNotificationService.js

Main responsibility: High-level notification events.

It chooses who receives which template for OTP, welcome, absent, leave, exam warning, and cancellation emails.

- Find the exported function names in this file.
- Find which controller calls those functions.
- Find which models are imported at the top.
- Find the validation errors thrown by the service.

aiService.js

Main responsibility: Backend gateway to AI service.

It adds timeouts, retries, error normalization, and keeps the frontend from talking directly to model endpoints.

- Find the exported function names in this file.
- Find which controller calls those functions.
- Find which models are imported at the top.
- Find the validation errors thrown by the service.

adminService.js

Main responsibility: Admin dashboard and destructive record cleanup.

It calculates admin metrics, sorts students by roll number, deletes users/classes, and cleans linked records.

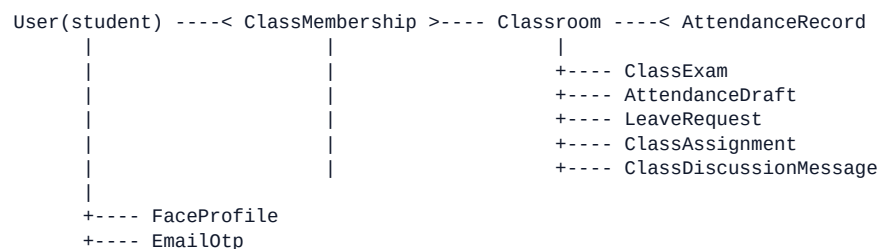
- Find the exported function names in this file.
- Find which controller calls those functions.
- Find which models are imported at the top.
- Find the validation errors thrown by the service.

Chapter 10: Database Models Explained

MongoDB stores data as documents. Mongoose models describe the expected fields for each kind of document. This chapter explains every important model in beginner language.

Model	What it stores	Used for
User	Student, teacher, and admin accounts. Stores role, name, user ID, roll number for students, password hash, email verification state, and profile data.	Authentication, dashboards, roster display, email notifications.
Classroom	A teacher-owned class with subject, code, section, room, schedule, join code, status, and archive summary.	Joining classes, dashboards, attendance, exams, and class lifecycle.
ClassMembership	The link between one student and one classroom.	Roster building, student class list, attendance summaries.
AttendanceRecord	Final attendance history after a manual, QR, AI, or cancelled class action.	Analytics, exports, student percentages, teacher summaries.
AttendanceDraft	Today Attendance Data. A temporary editable draft created from AI/manual attendance before final submission.	Correction window, early absence email, auto-finalization.
QrAttendanceSession	A short-lived QR token for a class attendance session.	Student scans QR; backend validates token and membership.
FaceProfile	Metadata about a student's enrolled face profile and model information.	Student enrollment status and AI attendance readiness.
ClassExam	Upcoming exam date and required attendance percentage for one class.	Eligibility alerts, calendar UI, attendance calculator.
LeaveRequest	Student uploaded proof/reason for absence, with teacher approve/reject state.	Medical reports, leave proof, and teacher review.
ClassAssignment	Teacher-created assignment with instructions, deadline, and attachments.	Class workspace learning material.
ClassAssignmentSubmission	Student assignment submission with files and comments.	Submission tracking.
ClassDiscussionMessage	Class discussion posts, replies, likes, and dislikes.	Class communication.
EmailOtp	Hashed OTP records for email verification, password reset, and profile email update.	Secure account activation and recovery.
EmailDeliveryLog	Status record for every important email attempt.	Debugging and audit trail for notifications.

Entity relationship diagram



EmailDeliveryLog is linked by notification metadata rather than one fixed relation.
QrAttendanceSession belongs to one Classroom and expires after use/time.

User model

The User model contains students, teachers, and admins. The role field decides which type of user it is. A student has a roll number. Teachers have profile fields such as designation and specialization. Passwords are stored as passwordHash, not plain text. This is important: the system should never store a readable password.

Classroom and membership

Classroom stores one teacher-owned class. ClassMembership stores which students joined that class. This is better than storing all student objects directly inside Classroom because membership can be queried independently. For example, to list all classes for one student, the backend can search ClassMembership by studentUserId.

AttendanceRecord vs AttendanceDraft

AttendanceRecord is final history. AttendanceDraft is temporary Today Attendance Data. The draft exists because AI can make mistakes. A photo result should not immediately become permanent attendance. Instead, it becomes editable. Students marked absent are emailed early, the teacher can correct the list, and final records are written only after final submission or auto-finalization.

OTP and email logs

EmailOtp stores hashed OTP records and expiration times. OTP records have a purpose such as email-verification or password-reset. EmailDeliveryLog stores whether an email was sent, skipped, or failed. This is very useful for debugging because email systems fail for network, SMTP, credential, or duplicate-notification reasons.

Why MongoDB is acceptable here

Attendance systems can be built on SQL too. PostgreSQL would be excellent for strict relational data and complex reports. MongoDB is acceptable in this project because many records are document-like and can evolve: class schedules, archive summaries, email metadata, attachment arrays, AI notes, and dashboard objects. Mongoose still provides schema structure so the database does not become uncontrolled.

Chapter 11: Model By Model Deep Dive

User

What it stores: Student, teacher, and admin accounts. Stores role, name, user ID, roll number for students, password hash, email verification state, and profile data.

Why it exists: Authentication, dashboards, roster display, email notifications.

- role controls whether the user is student, teacher, or admin.
- userId is uppercase and unique, so login can find one account.
- passwordHash stores the bcrypt hash instead of the original password.
- emailVerified and emailVerificationRequired protect login until OTP verification.
- rollNumber is required only for student accounts.

Classroom

What it stores: A teacher-owned class with subject, code, section, room, schedule, join code, status, and archive summary.

Why it exists: Joining classes, dashboards, attendance, exams, and class lifecycle.

- This model keeps one feature's data separate so it can be queried and maintained clearly.
- Indexes are used where repeated lookup is expected, such as by class, student, status, or date.
- The backend service sanitizes this data before sending it to the frontend.

ClassMembership

What it stores: The link between one student and one classroom.

Why it exists: Roster building, student class list, attendance summaries.

- This model keeps one feature's data separate so it can be queried and maintained clearly.
- Indexes are used where repeated lookup is expected, such as by class, student, status, or date.
- The backend service sanitizes this data before sending it to the frontend.

AttendanceRecord

What it stores: Final attendance history after a manual, QR, AI, or cancelled class action.

Why it exists: Analytics, exports, student percentages, teacher summaries.

- One final record belongs to one student, one class, and one session/date.
- status can be present, absent, late, or cancelled.
- verificationMethod tells how the record was created, such as manual, QR, or face recognition.
- source explains whether the record came from AI auto, teacher confirmation, manual add, or cancellation.
- These records power dashboard percentages and exports.

AttendanceDraft

What it stores: Today Attendance Data. A temporary editable draft created from AI/manual attendance before final submission.

Why it exists: Correction window, early absence email, auto-finalization.

- Draft records are editable and temporary.
- They store status, AI confidence, source, notes, and absentee email timestamp.
- A draft can be finalized by teacher or automatically after the configured window.
- Drafts make AI safer because the teacher can correct mistakes before permanent records.

QrAttendanceSession

What it stores: A short-lived QR token for a class attendance session.

Why it exists: Student scans QR; backend validates token and membership.

- This model keeps one feature's data separate so it can be queried and maintained clearly.
- Indexes are used where repeated lookup is expected, such as by class, student, status, or date.
- The backend service sanitizes this data before sending it to the frontend.

FaceProfile

What it stores: Metadata about a student's enrolled face profile and model information.

Why it exists: Student enrollment status and AI attendance readiness.

- Stores metadata about enrollment, not the full AI profile logic in MongoDB.
- faceModel and executionMode help detect stale or incompatible enrollments.
- averageQualityScore helps show whether enrollment quality was good.
- lastEnrolledAt tells when the student last completed enrollment.

ClassExam

What it stores: Upcoming exam date and required attendance percentage for one class.

Why it exists: Eligibility alerts, calendar UI, attendance calculator.

- Stores examDate and requiredAttendancePercentage.
- Only one active exam rule exists per class because of a partial unique index.
- Student dashboards use it to calculate eligibility and minimum classes to attend.
- Teacher dashboards use it to count eligible, recoverable, and at-risk students.

LeaveRequest

What it stores: Student uploaded proof/reason for absence, with teacher approve/reject state.

Why it exists: Medical reports, leave proof, and teacher review.

- This model keeps one feature's data separate so it can be queried and maintained clearly.
- Indexes are used where repeated lookup is expected, such as by class, student, status, or date.
- The backend service sanitizes this data before sending it to the frontend.

ClassAssignment

What it stores: Teacher-created assignment with instructions, deadline, and attachments.

Why it exists: Class workspace learning material.

- This model keeps one feature's data separate so it can be queried and maintained clearly.
- Indexes are used where repeated lookup is expected, such as by class, student, status, or date.
- The backend service sanitizes this data before sending it to the frontend.

ClassAssignmentSubmission

What it stores: Student assignment submission with files and comments.

Why it exists: Submission tracking.

- This model keeps one feature's data separate so it can be queried and maintained clearly.
- Indexes are used where repeated lookup is expected, such as by class, student, status, or date.
- The backend service sanitizes this data before sending it to the frontend.

ClassDiscussionMessage

What it stores: Class discussion posts, replies, likes, and dislikes.

Why it exists: Class communication.

- This model keeps one feature's data separate so it can be queried and maintained clearly.
- Indexes are used where repeated lookup is expected, such as by class, student, status, or date.
- The backend service sanitizes this data before sending it to the frontend.

EmailOtp

What it stores: Hashed OTP records for email verification, password reset, and profile email update.

Why it exists: Secure account activation and recovery.

- This model keeps one feature's data separate so it can be queried and maintained clearly.
- Indexes are used where repeated lookup is expected, such as by class, student, status, or date.
- The backend service sanitizes this data before sending it to the frontend.

EmailDeliveryLog

What it stores: Status record for every important email attempt.

Why it exists: Debugging and audit trail for notifications.

- This model keeps one feature's data separate so it can be queried and maintained clearly.
- Indexes are used where repeated lookup is expected, such as by class, student, status, or date.
- The backend service sanitizes this data before sending it to the frontend.

Chapter 12: Sample Data Documents

This chapter shows simplified sample documents. They are not copied from your database and contain no secrets. They help you understand what MongoDB stores.

Sample User document

```
{
  "_id": "66500000000000000000000000000001",
  "role": "student",
  "firstName": "Asha",
  "lastName": "Kumar",
  "userId": "STU-1001",
  "rollNumber": "CSE-041",
  "passwordHash": "<bcrypt-hash>",
  "email": "student@example.com",
  "emailVerified": true,
  "emailVerificationRequired": true,
  "department": "Computer Science",
  "batch": "2022-2026"
}
```

Sample Classroom document

```
{
  "_id": "665000000000000000000000000000101",
  "teacherUserId": "TCH-1001",
  "teacherName": "Dr. Meera Singh",
  "subjectName": "Artificial Intelligence",
  "subjectCode": "CSE-AI-301",
  "section": "A",
  "room": "Lab 2",
  "joinCode": "AI301A",
  "status": "active",
  "scheduleSlots": [
    { "day": "mon", "startTime": "10:00", "endTime": "11:00" },
    { "day": "wed", "startTime": "10:00", "endTime": "11:00" }
  ]
}
```

Sample ClassMembership document

```
{
  "_id": "665000000000000000000000000000201",
  "classId": "665000000000000000000000000000101",
  "studentUserId": "STU-1001",
  "studentName": "Asha Kumar",
  "rollNumber": "CSE-041"
}
```

Sample AttendanceDraft document

```
{
  "_id": "665000000000000000000000000000301",
  "classId": "665000000000000000000000000000101",
  "teacherUserId": "TCH-1001",
  "dateKey": "2026-05-20",
  "status": "draft",
  "records": [
    {
      "studentId": "STU-1001",
      "studentName": "Asha Kumar",
      "rollNumber": "CSE-041",
      "status": "present",
      "source": "ai-suggested",
      "confidence": 0.86
    },
    {
      "studentId": "STU-1002",
      "studentName": "Ravi Patel",
      "rollNumber": "CSE-042",
      "status": "absent",
      "source": "ai-suggested"
    }
  ]
}
```

```
    }  
  ],  
  "absenteeEmailSentAt": "2026-05-20T10:10:00.000Z",  
  "expiresAt": "2026-05-20T22:10:00.000Z"  
}
```

Sample final AttendanceRecord document

```
{  
  "_id": "66500000000000000000000000401",  
  "sessionId": "CSE-AI-301-2026-05-20",  
  "teacherUserId": "TCH-1001",  
  "studentId": "STU-1001",  
  "studentName": "Asha Kumar",  
  "rollNumber": "CSE-041",  
  "classId": "66500000000000000000000000101",  
  "className": "Artificial Intelligence",  
  "status": "present",  
  "verificationMethod": "face-recognition",  
  "source": "teacher-confirmed",  
  "confidence": 0.86,  
  "recordedAt": "2026-05-20T10:15:00.000Z"  
}
```

Sample ClassExam document

```
{  
  "_id": "66500000000000000000000000501",  
  "classId": "66500000000000000000000000101",  
  "teacherUserId": "TCH-1001",  
  "title": "Mid Semester Exam",  
  "examDate": "2026-06-10T00:00:00.000Z",  
  "requiredAttendancePercentage": 75,  
  "status": "active"  
}
```

Sample LeaveRequest document

```
{  
  "_id": "66500000000000000000000000601",  
  "classId": "66500000000000000000000000101",  
  "teacherUserId": "TCH-1001",  
  "studentUserId": "STU-1001",  
  "studentName": "Asha Kumar",  
  "requestType": "medical",  
  "absenceDate": "2026-05-18T00:00:00.000Z",  
  "reason": "Medical appointment",  
  "attachments": [  
    {  
      "fileName": "medical-report.pdf",  
      "fileType": "application/pdf",  
      "fileSize": 120000,  
      "dataUrl": "data:application/pdf;base64,<omitted>"  
    }  
  ],  
  "status": "pending"  
}
```

Sample EmailOtp document

```
{  
  "_id": "66500000000000000000000000701",  
  "role": "student",  
  "userId": "STU-1001",  
  "email": "student@example.com",  
  "purpose": "password-reset",  
  "otpHash": "<bcrypt-hash-of-otp>",  
  "attempts": 0,  
  "consumedAt": null,  
  "expiresAt": "2026-05-20T10:20:00.000Z"  
}
```

Chapter 13: Data Lifecycle Examples

A data lifecycle explains how one piece of data is born, changes, and gets used later. Understanding lifecycle is more powerful than memorizing fields.

Lifecycle of a student account

Signup form data

- > User document created with passwordHash and emailVerified=false
- > EmailOtp document created for email-verification
- > OTP verified and consumed
- > User emailVerified=true
- > Student joins classes through ClassMembership
- > Student enrolls face profile through FaceProfile and AI enrollment store
- > Student accumulates AttendanceRecord documents
- > Student dashboard summarizes records into useful analytics

Lifecycle of one attendance day

Teacher starts attendance

- > AI/manual/QR source creates attendance information
- > AI path creates AttendanceDraft first
- > Absentee email notification is sent from draft
- > Teacher edits draft if needed
- > Final submission writes AttendanceRecord documents
- > Attendance analytics recompute percentages
- > Student dashboard and teacher dashboard show updated data
- > Exam eligibility may change because present/total counts changed

Lifecycle of an exam rule

Teacher sets exam date and required percentage

- > ClassExam document saved or updated
- > Student dashboard fetch includes upcoming exam
- > examScheduleService counts remaining scheduled classes
- > Eligibility object is created for each student
- > Email warning can be sent if attendance is below requirement
- > Teacher dashboard summarizes eligible/recoverable/at-risk students

Lifecycle of a leave request

Student submits reason and document

- > LeaveRequest status=pending
- > Teacher views attachment in class workspace
- > Teacher approves or rejects
- > LeaveRequest status changes and teacherNote is saved
- > Student receives email notification
- > Student class detail shows updated status

Lifecycle of an email

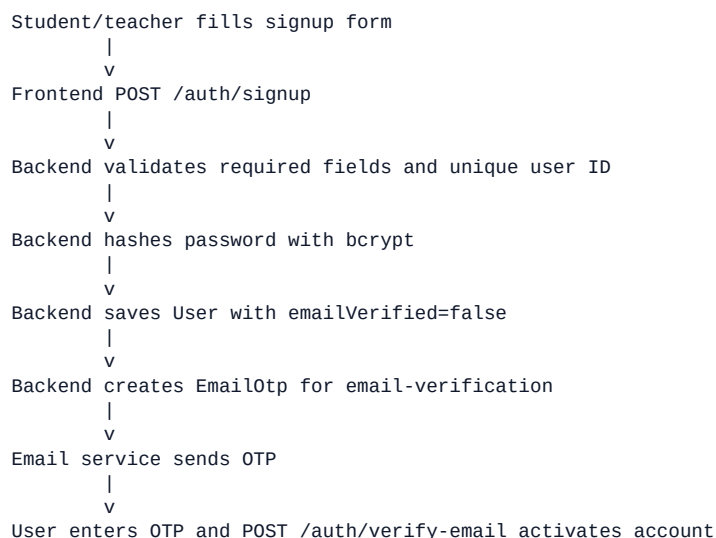
Feature triggers notification

- > emailNotificationService picks recipient and template
- > emailTemplateService builds professional subject/text/html
- > emailService checks duplicate notificationKey if present
- > email job enters queue
- > Nodemailer sends through SMTP with timeout and retry
- > EmailDeliveryLog records sent, skipped, or failed

Chapter 14: Authentication, OTP, And Password Reset

Authentication is the process of proving who the user is. SAMS uses role, user ID, password, and email OTP verification. A student and teacher can register. Admin is seeded by the backend. Email verification is required before login for accounts that need it.

Signup flow



Password hashing

When a user creates a password, the backend hashes it using bcrypt. A hash is one-way. During login, the backend hashes the submitted password in a comparable way and checks whether it matches the stored hash. This protects users if the database is leaked because attackers do not immediately see the original passwords.

OTP cooldown

The OTP system has a resend cooldown. The frontend shows a timer before the resend button can be used again, and the backend also blocks early OTP creation. This matters because frontend-only timers can be bypassed. Real protection belongs on the backend.

Password reset flow

1. User opens Forgot Password.
2. User selects student/teacher and enters account ID.
3. Backend sends password reset OTP.
4. User must wait 60 seconds before resend.
5. User enters OTP.
6. Backend verifies OTP and returns a short-lived reset session token.
7. User enters new password twice.
8. Backend checks both passwords match and hashes the new password.
9. User can log in with the new password.

Why not reset password immediately after OTP request

Requesting OTP only proves that a code was sent. It does not prove the user owns the mailbox until the user enters the correct OTP. That is why the flow separates request, verify, and password update. This also gives the UI a clearer beginner-friendly sequence.

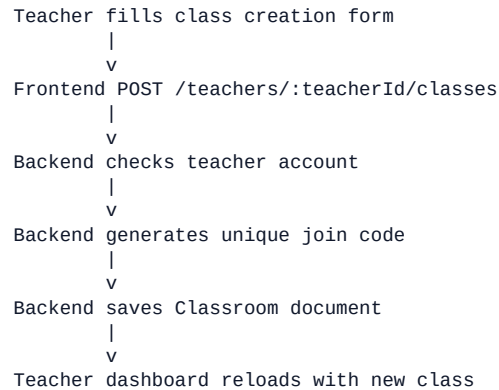
Admin account

The backend creates or updates a default admin account during startup. Admin login is separate in the UI and does not show student/teacher signup or forgot password links. In a production system, default admin credentials should be changed and protected with stronger access controls.

Chapter 15: Class Management

Class management begins with the teacher. A teacher creates a classroom with subject name, code, section, room, schedule, academic details, and a join code. Students use the join code to become members.

Class creation flow



Join code flow

A join code is a simple way for students to join the correct class. The student enters the code, the backend normalizes it, finds the Classroom, checks that the student exists, and creates a ClassMembership. A unique index prevents the same student from joining the same class multiple times.

Roster management

Teachers can add, update, or remove students in their class workspace. The backend checks that the class belongs to the teacher before changing the roster. This prevents one teacher from editing another teacher's class.

Archiving a class

When a semester or class is over, the teacher can archive it. Archiving saves summary data such as student count, total sessions, average attendance, flagged students, and latest session date. Archived classes become inactive for attendance actions. This protects old records while keeping history visible.

Why archive instead of delete

Delete removes data. Archive preserves history but stops new actions. For academic records, archiving is usually safer. Admin still has delete controls for incorrect or unwanted data, but normal class lifecycle should use archive.

Chapter 16: Attendance Workflows

Attendance is the core feature. SAMS supports manual attendance, QR attendance, AI photo attendance, cancellation, drafts, finalization, and export. The important idea is that final attendance should be trustworthy, not just fast.

Manual attendance

Manual attendance is the fallback and control method. A teacher selects present, absent, or late for each roster student and submits. The backend writes AttendanceRecord documents. Manual attendance is useful when AI is not available, QR failed, or the teacher wants direct control.

QR attendance

```

Teacher creates QR session
  |
  v
Backend creates QrAttendanceSession with token and expiry
  |
  v
Frontend turns token URL into QR image
  |
  v
Student scans QR
  |
  v
Backend validates token, expiry, class, and membership
  |
  v
AttendanceRecord is saved for that student

```

AI classroom photo attendance

AI attendance is not final automatically. The teacher captures or uploads a classroom photo. The frontend compresses/resizes the image to keep payload size manageable. The backend sends roster and image data to the AI service. The AI service detects faces, matches them only against students in that class roster, and returns suggested present students, unknown faces, review candidates, and absent students.

Today Attendance Data

```

AI result from classroom photo
  |
  v
Backend creates AttendanceDraft for today's class
  |
  v
Draft records show present/absent/late students
  |
  v
Absent students receive email early
  |
  v
Teacher can add/remove/change students in draft
  |
  v
Teacher clicks Final Submission
  |
  v
Backend writes final AttendanceRecord documents

```

Why absent email is sent before finalization

The early absent email is a correction mechanism. If a student was actually present but AI missed them, the student can tell the teacher before final submission. This is better than silently saving a wrong final record. The teacher still controls the final list.

Auto finalization after 12 hours

The attendance draft scheduler checks old drafts. If a draft stays open too long, it can be automatically finalized. This prevents Today's Attendance Data from staying pending forever. In a real institution, the auto-finalization time can be changed according to policy.

Cancelled class

If a class is cancelled, the backend records cancelled status and sends class cancellation email. Cancelled attendance should not hurt a student's percentage because it is not a class the student missed. This distinction is handled in attendance analytics by ignoring cancelled records for the student's attended/total count.

Attendance formula

```
attendance percentage = (present_or_late_count / total_count) * 100
```

Example:

```
present_or_late_count = 18
```

```
total_count = 24
```

```
percentage = (18 / 24) * 100 = 75%
```

Cancelled class is not counted as total_count.

Absent class is counted in total_count but not present_or_late_count.

Finalization checklist

- Make sure the roster is correct before attendance.
- Check AI suggested present students.
- Check review candidates and unknown detections.
- Add students who were present but missed by AI.
- Remove students wrongly suggested as present.
- Read relevant verification notes, especially missing enrollment or low confidence notes.
- Finalize only when the draft represents the real class.

Chapter 17: Feature By Feature Deep Dive

This chapter explains each major feature as a complete story. For every feature, focus on goal, files, steps, and edge cases. That is how developers understand large systems.

Student registration and email verification

Goal: Create a real account only after the student or teacher proves ownership of the email address.

Important files: AuthPage.jsx, SignupForm.jsx, authController.js, authService.js, emailOtpService.js, emailNotificationService.js, User.js, EmailOtp.js.

- Frontend collects role, name, ID, password, email, and profile fields.
- Backend validates required fields and checks unique user ID and student roll number.
- Password is hashed with bcrypt before saving.
- User is saved with emailVerified=false.
- OTP is generated, hashed, saved, and emailed.
- User enters OTP; backend verifies hash and expiry; account becomes verified.

Edge case to remember: If SMTP is down, the account may be created but the email status will show error or skipped. In development, devOtp may be exposed if configured, but production should disable that.

Beginner checkpoint

After reading this feature, try to say out loud which frontend page starts it, which backend service owns it, and which MongoDB model stores the result.

Login

Goal: Allow only the correct role and credentials into the correct dashboard.

Important files: LoginForm.jsx, AuthPage.jsx, session.js, authController.js, authService.js, User.js.

- User selects role and enters ID and password.
- Backend searches User by role and normalized userId.
- Backend blocks login if verification is required and email is not verified.
- bcrypt compares submitted password with stored hash.
- Frontend stores the returned user session and routes to student, teacher, or admin dashboard.

Edge case to remember: If a student tries teacher role with the same ID, login fails because role is part of the search.

Beginner checkpoint

After reading this feature, try to say out loud which frontend page starts it, which backend service owns it, and which MongoDB model stores the result.

Forgot password with OTP

Goal: Recover access without exposing old passwords or allowing unlimited OTP spam.

Important files: ResetPasswordForm.jsx, AuthPage.jsx, authController.js, authService.js, emailOtpService.js, EmailOtp.js.

- User requests reset OTP for role and account ID.
- Backend creates password-reset OTP and sends email.
- Frontend disables resend for 60 seconds.
- Backend also enforces resend cooldown.
- User verifies OTP and receives a short-lived reset session token.
- User enters new password twice; backend checks match and hashes new password.

Edge case to remember: The reset token prevents the app from keeping the OTP reusable during the password entry step.

Beginner checkpoint

After reading this feature, try to say out loud which frontend page starts it, which backend service owns it, and which MongoDB model stores the result.

Teacher class creation

Goal: Let teachers create structured classes students can join.

Important files: TeacherClassCreationSection.jsx, classroomController.js, classroomService.js, Classroom.js.

- Teacher fills subject, code, section, room, batch, semester, and schedule slots.
- Backend verifies teacher account.
- Backend generates a unique join code.
- Classroom document is saved.
- Teacher dashboard reloads and displays the class.

Edge case to remember: Schedule data is normalized so later calculations, such as exam classes before date, can work reliably.

Beginner checkpoint

After reading this feature, try to say out loud which frontend page starts it, which backend service owns it, and which MongoDB model stores the result.

Student joins class

Goal: Connect a student account to a teacher classroom using a join code.

Important files: JoinClassPage.jsx, StudentClassroomSection.jsx, classroomController.js, classroomService.js, ClassMembership.js.

- Student enters join code.
- Backend finds active Classroom by normalized join code.
- Backend verifies student exists.
- ClassMembership is created with classId and studentUserId.
- Student dashboard now includes the class.

Edge case to remember: A unique membership index prevents duplicate joins.

Beginner checkpoint

After reading this feature, try to say out loud which frontend page starts it, which backend service owns it, and which MongoDB model stores the result.

Face enrollment

Goal: Build a reusable student face profile for future AI attendance.

Important files: `StudentFaceEnrollmentPage.jsx`, `faceProfileController.js`, `faceProfileService.js`, `aiService.js`, `enrollment_pipeline.py`, `face_recognition_service.py`.

- Student uploads several reference images.
- Backend validates student account and forwards images to AI service.
- AI rejects unusable images and creates embeddings for valid single-face images.
- AI averages embeddings and stores enrollment profile.
- Backend saves `FaceProfile` metadata for dashboard readiness.

Edge case to remember: If the model changes later, old embeddings may be incompatible and re-enrollment is required.

Beginner checkpoint

After reading this feature, try to say out loud which frontend page starts it, which backend service owns it, and which MongoDB model stores the result.

AI classroom attendance

Goal: Suggest present students from a classroom image without making final records immediately.

Important files: TeacherClassroomPage.jsx, teacherClassroomService.js, aiService.js, classroom_attendance_pipeline.py.

- Teacher captures/uploads classroom photo.
- Frontend compresses and sends image to backend.
- Backend builds class roster payload for AI.
- AI detects faces, creates tracks, compares against roster profiles, and returns suggestions.
- Backend creates Today Attendance Data draft.
- Absentees receive early email notification.

Edge case to remember: Students not enrolled or faces with low confidence go to notes/review instead of automatic final attendance.

Beginner checkpoint

After reading this feature, try to say out loud which frontend page starts it, which backend service owns it, and which MongoDB model stores the result.

Today Attendance Data correction

Goal: Give teacher a correction window between AI suggestion and final permanent attendance.

Important files: TeacherClassroomPage.jsx, teacherClassroomService.js, AttendanceDraft.js, AttendanceRecord.js.

- Teacher reviews draft list.
- Teacher changes absent/present/late status when needed.
- Teacher saves draft updates.
- Teacher finalizes or scheduler finalizes after configured time.
- Backend writes final AttendanceRecord documents.

Edge case to remember: Drafts make wrong AI suggestions recoverable before they affect analytics.

Beginner checkpoint

After reading this feature, try to say out loud which frontend page starts it, which backend service owns it, and which MongoDB model stores the result.

Manual attendance

Goal: Allow teacher-controlled attendance without AI or QR dependency.

Important files: TeacherClassroomPage.jsx, teacherClassroomService.js, AttendanceRecord.js.

- Teacher marks each roster student.
- Frontend submits statuses and notes.
- Backend validates ownership and active class.
- AttendanceRecord documents are saved.
- Dashboards update through attendance analytics.

Edge case to remember: Manual attendance is the emergency fallback if camera, AI, or QR fails.

Beginner checkpoint

After reading this feature, try to say out loud which frontend page starts it, which backend service owns it, and which MongoDB model stores the result.

QR attendance

Goal: Let students self-mark attendance through a short-lived scan token.

Important files: TeacherClassroomPage.jsx, QrAttendancePage.jsx, teacherClassroomService.js, QrAttendanceSession.js.

- Teacher creates QR session.
- Backend saves token and expiry.
- Frontend renders QR code with scan URL.
- Student scans and submits identity.
- Backend verifies token, expiry, and membership before saving attendance.

Edge case to remember: Expired or closed QR tokens should not create records.

Beginner checkpoint

After reading this feature, try to say out loud which frontend page starts it, which backend service owns it, and which MongoDB model stores the result.

Leave proof upload and review

Goal: Let students explain absence with documents and let teachers approve or reject.

Important files: StudentClassDetailPage.jsx, TeacherClassroomPage.jsx, leaveRequestService.js, LeaveRequest.js, emailNotificationService.js.

- Student selects class, date, type, reason, and attachment.
- Backend verifies student membership.
- LeaveRequest is saved with pending status.
- Teacher opens class workspace and views attachment online.
- Teacher approves/rejects with note.
- Student receives email status update.

Edge case to remember: Attachment size and file data should be controlled to protect database size.

Beginner checkpoint

After reading this feature, try to say out loud which frontend page starts it, which backend service owns it, and which MongoDB model stores the result.

Exam rule and eligibility

Goal: Convert attendance and schedule into eligibility and recovery advice.

Important files: TeacherExamsSection.jsx, StudentExamSection.jsx, examScheduleService.js, ClassExam.js.

- Teacher sets exam title, date, required percentage, and note.
- Backend validates date and percentage.
- Dashboards count scheduled classes before exam.
- Backend calculates current eligibility and minimum classes to attend.
- Students see action text and warnings.

Edge case to remember: If no scheduled classes remain, the system tells the student to talk to the teacher instead of pretending recovery is possible.

Beginner checkpoint

After reading this feature, try to say out loud which frontend page starts it, which backend service owns it, and which MongoDB model stores the result.

Class cancellation

Goal: Record that a class did not happen and notify students.

Important files: `TeacherClassroomPage.jsx`, `teacherClassroomService.js`, `emailNotificationService.js`, `AttendanceRecord.js`.

- Teacher cancels today's class with a reason.
- Backend writes cancelled records or cancellation status.
- Email notification is sent to class students.
- Attendance analytics do not count cancelled class against students.

Edge case to remember: Cancelled is different from absent. A student should not lose percentage for a class that was not held.

Beginner checkpoint

After reading this feature, try to say out loud which frontend page starts it, which backend service owns it, and which MongoDB model stores the result.

Class archive

Goal: Freeze a completed class while keeping its history.

Important files: TeacherClassesPanel.jsx, teacherClassroomService.js, Classroom.js, teacherDashboardService.js.

- Teacher clicks End Class.
- Backend computes archive summary from attendance and roster.
- Classroom status becomes archived.
- Active attendance operations are blocked.
- Dashboard still shows historical summary.

Edge case to remember: Archive is normal lifecycle. Delete should be reserved for admin cleanup or wrong data.

Beginner checkpoint

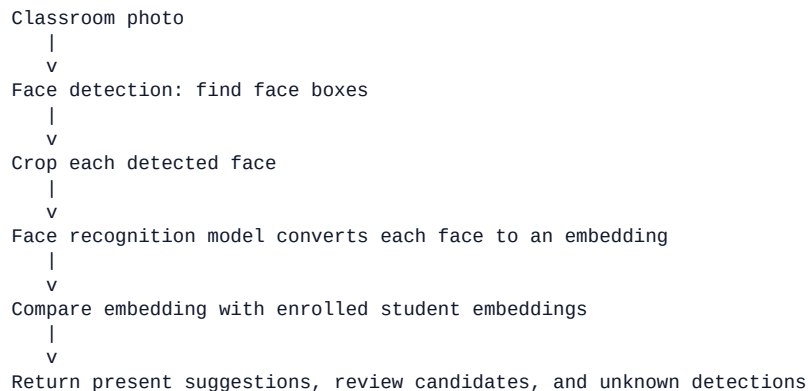
After reading this feature, try to say out loud which frontend page starts it, which backend service owns it, and which MongoDB model stores the result.

Chapter 18: AI Service And Face Recognition

The AI service is the most technical part of SAMS, but it can be understood step by step. The AI does not magically know names from a photo. It compares the face in a classroom photo with face profiles that students enrolled earlier.

Core idea: detection then recognition

Face detection asks: where are the faces in this image? Face recognition asks: whose face is this? These are different tasks. A detector can find a face without knowing the person's identity. A recognizer needs enrolled references to compare against.



What is an embedding

An embedding is a list of numbers that represents important features of a face. You do not read the numbers directly. The model creates them. If two embeddings are close, the faces are probably the same person. If they are far apart, they are probably different people. SAMS uses cosine similarity to compare embeddings.

Selected model stack

Task	Selected model	Reason
Face detection	RetinaFace/SCRFD-10GF from InsightFace pack	Strong multi-face detection and widely used in modern face analysis pipelines.
Face tracking	Embedding centroid tracker	Groups repeated detections across captures using embedding similarity.
Face recognition	ArcFace ResNet100@Glint360K from antelopev2	Produces strong 512-dimensional identity embeddings for face matching.
Fallback	buffalo_l pack with ArcFace ResNet50@WebFace600K	Keeps service usable when preferred antelopev2 files are missing.

Why ArcFace

ArcFace is designed for face verification and recognition. It learns an embedding space where faces of the same person are close and faces of different people are separated. For this project, that is exactly what is needed. The system does not need to train a new model for every class. It only needs to store student reference embeddings and compare classroom detections against them.

Model alternatives

Alternative	What it is	Why not primary choice here
OpenCV Haar cascades	Old face detection method based on hand-designed features.	Fast but weak in varied classroom angles and lighting. It detects faces, not identity.

Alternative	What it is	Why not primary choice here
OpenCV LBPH	Traditional local texture face recognition.	Useful for simple controlled images but not strong enough for modern classroom photos.
Dlib face recognition	Popular 128-dimensional face embeddings.	Good learning option, but InsightFace/ArcFace generally offers stronger modern performance.
FaceNet	Deep learning embedding approach.	Good alternative, but current code and metadata are centered on ArcFace embedding compatibility.
DeepFace wrapper	Python wrapper around multiple face models.	Convenient, but less direct control over production thresholds and model readiness.
Custom CNN	Train your own neural network.	Requires large labeled dataset, training infrastructure, evaluation, and fairness testing. Not practical for this project.

Enrollment flow

```

Student uploads 6-10 face images
    |
    v
AI service checks each image
    |
    +--> reject if no face
    +--> reject if multiple faces unless configured
    +--> reject if quality too low
    |
    v
Each usable face becomes an embedding
    |
    v
Average embedding is calculated
    |
    v
Profile is stored with model name and execution mode

```

Why require multiple enrollment images

One photo is fragile. Lighting, angle, blur, glasses, or expression can change the embedding. Multiple reference images make the average embedding more stable. The backend asks for more images than the minimum because some images may be rejected for quality.

Recognition thresholds

The AI service uses thresholds from `ai-service/app/core/config.py`. Recognition threshold decides when a match is strong enough to suggest present. Review threshold allows lower-confidence matches to appear in manual review. Recognition minimum margin helps avoid accepting a match when the best and second-best students are too close.

Why teacher review remains necessary

AI can fail. A face can be hidden, blurry, turned away, or confused with a similar-looking person. Lighting can reduce quality. A student may not be enrolled. That is why SAMS treats AI output as a suggestion. The teacher still confirms final attendance.

Model compatibility

Embeddings from different models should not be mixed. A FaceNet embedding and an ArcFace embedding do not live in the same vector space. The AI service stores model metadata with the profile and checks compatibility. If the profile was created with an older or different model, the student should re-enroll.

Privacy and ethics

Face recognition is sensitive. A real deployment should ask for consent, explain data use, protect stored embeddings, limit access, and define deletion policy. The project stores metadata and local AI profile data for

development, but a production system should use stronger storage protection, audit logs, and institutional approval.

Chapter 19: AI Algorithm Details For Beginners

This chapter explains the AI logic more slowly. You do not need to know advanced mathematics to understand the pipeline, but you should know what each term means.

Step 1: image loading

The AI service receives images as references, often data URLs. It must decode the image, check size limits, and load it into a format the model can process. This step can fail if the file is not an image, too large, corrupt, or unreachable.

Step 2: face detection

The detector scans the image and returns bounding boxes for faces. A bounding box is a rectangle around a face. Detection quality matters because recognition only works on the cropped face. If the detector misses a face, the recognizer never gets a chance to identify that student.

Step 3: face crop and quality

After detection, the service crops the face region, often with a margin around it. It also calculates quality signals. Blurry, tiny, side-facing, or unclear faces can be rejected or flagged. This protects the system from confidently matching bad evidence.

Step 4: embedding creation

The recognition model converts a face crop into a numerical vector. In this project, the embedding dimension is configured as 512. A 512-dimensional embedding is not a picture. It is a mathematical identity representation created by the neural network.

Step 5: similarity comparison

To compare two faces, the service compares their embeddings. Cosine similarity is commonly used. A score closer to 1 means more similar. A score closer to 0 means less similar. The project combines average embedding similarity and reference embedding similarity for more stable matching.

Step 6: threshold and margin

A threshold says how high the similarity must be before the system accepts a match. A margin says the best match must be clearly better than the second-best match. Without a margin, two similar students could cause a wrong automatic match.

Step 7: one-student-one-track

If multiple detected faces match the same enrolled student, only the strongest track can be automatically suggested for that student. Other detections go to manual review. This avoids marking one student present multiple times.

Step 8: notes and review queue

Good AI systems explain uncertainty. SAMS returns notes such as missing face profiles, re-enrollment required, no faces detected, low confidence, or manual review. The UI filters noisy notes and shows useful ones so the teacher can act.

Where thresholds live

Setting	Meaning
RECOGNITION_THRESHOLD	Minimum confidence for automatic recognition.
RECOGNITION_MIN_MARGIN	How much better the best match must be than the next match.
REVIEW_THRESHOLD	Lower threshold where a possible match can go to manual review.
ENROLLMENT_MIN_QUALITY_SCORE	Minimum quality for an enrollment image to be used.
LOW_QUALITY_FACE_THRESHOLD	Quality value below which detected face is flagged.
TRACKING_SIMILARITY_THRESHOLD	Similarity needed to group detections into one track.

How to explain the AI in one minute

Students first enroll their faces. The AI stores mathematical face representations called embeddings. Later, a teacher uploads a classroom photo. The AI detects faces in the photo, converts each face to an embedding, compares those embeddings only against students in that class roster, and returns suggested present students. The teacher reviews and finalizes.

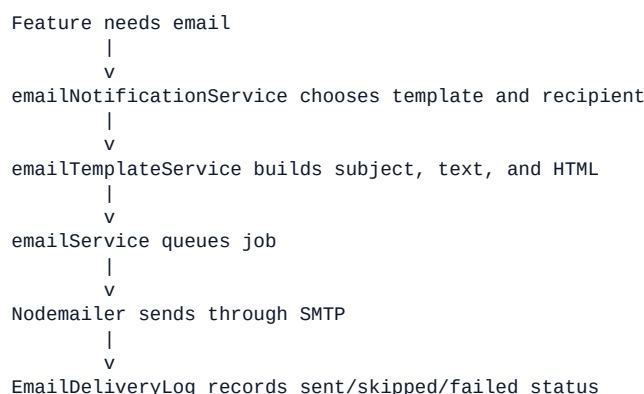
Chapter 20: Email Notification System

Email makes the project feel like a real system. Instead of only showing messages inside the dashboard, SAMS can notify users about OTPs, welcome messages, password reset, leave status, absence, exam warnings, and class cancellation.

SMTP in simple words

SMTP is the protocol used to send email. The backend connects to an SMTP server such as Gmail SMTP. Nodemailer is the Node.js library that creates the email transporter and sends messages. Gmail requires an app password, not the normal Gmail login password.

Email service architecture



Notification events

Event	Recipient	Why it matters
Signup verification OTP	Student or teacher	Activates account only after email ownership is proven.
Welcome email	Verified user	Confirms account is ready.
Password reset OTP	Student or teacher	Allows secure recovery.
Profile email OTP	New email address	Prevents accidental or fake profile email changes.
Leave approved/rejected	Student	Keeps student informed about uploaded absence proof.
Absent notification	Student	Allows quick correction before final attendance submission.
Exam warning	Student	Warns when attendance is below required exam percentage.
Class cancellation	Students in class	Prevents confusion about schedule changes.

Queue and retries

Sending email can be slow. The backend uses a delivery queue with limited concurrency so many emails do not overload the SMTP provider. It also retries transient failures. Delivery logs make it possible to see whether a message was sent, skipped because email was disabled, skipped because the recipient was missing, or failed because SMTP had an error.

SMTP timeout

SMTP timeout means the backend tried to connect or send through the mail server but did not get a response in time. It can happen because of weak network, blocked port, slow SMTP server, wrong SMTP configuration, or too short a timeout setting. It does not mean the whole app is broken; it means the email service needs attention.

Chapter 21: Exam Eligibility And Attendance Calculator

Exam eligibility turns attendance data into an action plan. Instead of only saying 'you have 68 percent', the system tells the student whether they are eligible, how many scheduled classes remain before the exam, and how many of those classes they must attend.

Teacher exam rule

A teacher sets an exam date and required attendance percentage, for example 75 percent. This creates a ClassExam record. Student and teacher dashboards both show upcoming exam information.

Eligibility formula

```
Inputs:
P = present count
T = total counted classes so far
R = required percentage / 100
F = scheduled classes before exam

Projected total at exam = T + F
Minimum classes to attend = ceil(R * (T + F) - P)

If minimum classes to attend <= F:
    student can still qualify by attending enough future classes
else:
    student cannot reach required percentage through normal scheduled classes
```

Worked example

Suppose a student has attended 18 out of 26 classes. Current attendance is 69 percent. The teacher requires 75 percent. There are 8 scheduled classes before the exam. Projected total is 34. Required attended classes at exam time is $\text{ceil}(0.75 * 34) = \text{ceil}(25.5) = 26$. The student already has 18, so they must attend 8 more. Since 8 classes remain, they can qualify only if they attend every remaining class.

Why schedule matters

Without the class schedule, the calculator would not know how many classes remain before the exam. That is why Classroom stores scheduleSlots. The exam service counts scheduled slots between now and the exam date.

Student-entered calculator

The student attendance calculator can also take an exam date and required percentage input from the student. It uses current attendance percentage and schedule-based future class count to estimate safe attendance requirements. This helps students plan even before the teacher creates the official exam rule.

Chapter 22: Student Journey

The student journey starts at signup and ends with useful awareness: joined classes, attendance analytics, exam eligibility, documents, notifications, and profile control.

Student signup to dashboard

```
Signup -> Email OTP verification -> Login -> Student dashboard
|
+--> Join class using code
+--> Enroll face profile
+--> Open class detail page
+--> Submit leave/medical document
+--> Check attendance analytics
+--> Check exam eligibility
+--> Read notifications
```

What the student dashboard answers

Question	Dashboard data used	Why useful
How many classes have I joined?	ClassMembership + Classroom	Shows academic scope.
What is my attendance in each class?	AttendanceRecord summaries	Identifies weak classes.
Am I safe for an exam?	ClassExam + attendance + schedule	Turns raw percentage into action.
How many classes must I attend?	Eligibility formula	Gives a concrete target.
Was I marked absent recently?	AttendanceRecord + notifications	Lets student act quickly.
Is my face profile ready?	FaceProfile	Shows whether AI attendance can recognize student.
What leave documents did I submit?	LeaveRequest	Tracks approval state.

Leave document upload

A student can submit a leave request for a class and attach a PDF/image style file as a data URL. The teacher sees the request in the class details section and can approve or reject it. The file is viewable online from stored data, so the teacher does not need to download it just to inspect it.

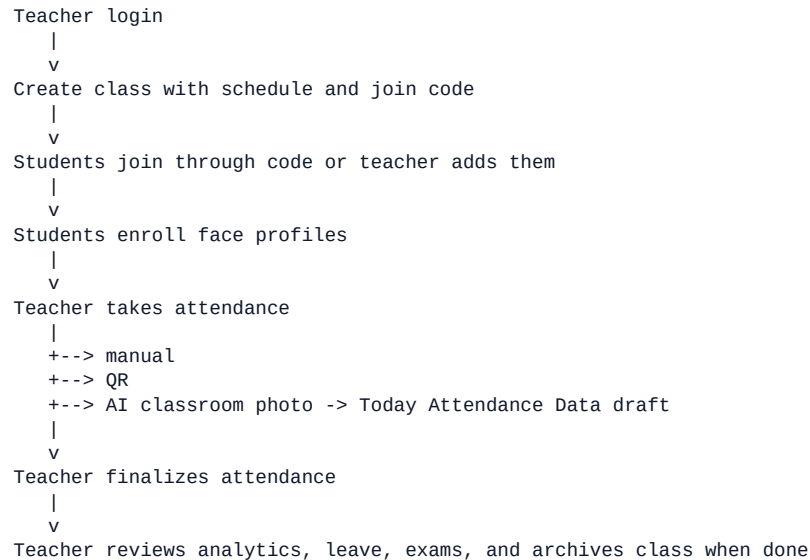
Notifications

Notifications are meant to pull attention to important things: low attendance, exam risk, absence, class changes, and pending profile work. A notification bell or notification page is useful because students should not hunt through many panels to find urgent information.

Chapter 23: Teacher Journey

The teacher journey is about control and correction. The teacher creates classes, gets students into rosters, takes attendance, reviews AI results, handles leave, sets exam rules, and archives completed classes.

Teacher class workflow



Class workspace

The teacher class workspace is the operational center. It loads classroom information, roster, overview metrics, today's attendance, active draft, recent sessions, leave requests, exam rule, QR status, and face profile readiness. This page is large because one class has many workflows.

Correction responsibility

The teacher must not blindly trust AI. The system helps by showing confidence, review queue, unknown detections, relevant verification notes, and draft editing controls. The final attendance should represent the teacher-reviewed truth, not only the model's guess.

End of semester

When a class is completed, the teacher archives it. Archived classes remain in historical views but active operations are blocked. This prevents accidental attendance submission into an old semester.

Chapter 24: Admin Controls

The admin panel is not for normal teaching work. It is for system control. Admin can remove incorrect classes, students, or teachers, and can toggle email verification state if required.

Why admin features are dangerous

Deleting a class or user can cascade into many related records: memberships, attendance, drafts, exams, leave requests, assignments, discussions, OTPs, and email logs. That is why admin actions should show confirmation dialogs and summarize deleted counts.

Admin dashboard should stay focused

The admin dashboard should not show teacher-specific cards like pending leave or student-specific cards like exam eligibility as primary metrics. Admin needs clean record counts, lists sorted predictably, and destructive actions with clear confirmation.

Production improvement

In production, admin access should be protected more strongly than the default project seed. Use a strong password, role-based middleware, audit logging for admin actions, and possibly two-factor authentication.

Chapter 25: Running And Deployment

Running a full-stack app means starting several services. In SAMS, the frontend, backend, AI service, and MongoDB may run separately or through Docker Compose.

Local development ports

Service	Typical local URL	What to check
Frontend	http://localhost:5173	Browser UI loads.
Backend	http://localhost:4000/api/v1/health	Health returns JSON.
AI service	http://localhost:8000/health	Model readiness and warnings.
MongoDB	mongodb://localhost:27017/sams	Backend can connect.

Environment variables

Environment variables keep runtime settings outside code. Examples include `MONGO_URI`, `AI_SERVICE_URL`, `EMAIL_HOST`, `EMAIL_USER`, `EMAIL_PASSWORD`, `EMAIL_OTP_EXPIRES_MINUTES`, and `FRONTEND_ORIGIN`. Never include real secrets in reports, Git commits, screenshots, or public chats.

Docker Compose

`docker-compose.yml` defines four services: `frontend`, `backend`, `ai-service`, and `mongo`. Compose is useful because it starts the whole stack with correct network names. The backend can call `ai-service` by container name instead of `localhost` when running inside Docker.

Health checks

The backend health endpoint checks not only whether Express is alive, but also database, AI readiness, and email service state. A service can be alive but degraded. For example, AI may be running but missing model files, or email may be configured but SMTP verification may time out.

Chapter 26: Testing And Debugging Guide

Debugging means finding where the flow stopped. Do not guess randomly. Follow the request path.

General debugging path

1. Is the frontend route loading?
2. Does browser console show an error?
3. Is the frontend calling the correct backend URL?
4. Does the backend route exist?
5. Does the controller return a clear error?
6. Does the service validation fail?
7. Is MongoDB connected?
8. If AI is involved, is AI /ready true?
9. If email is involved, is email status online?
10. Does the database contain the expected document?

Common problems

Problem	Likely cause	Where to check
Login fails	Wrong role, ID, password, or unverified email.	AuthPage, authService.js, User document.
OTP not received	SMTP config, timeout, spam folder, email disabled.	email/status, email logs, backend .env without printing secrets.
Student cannot join class	Wrong join code, class archived, duplicate membership.	classroomService.js, ClassMembership.
AI attendance empty	No face profiles, bad photo, AI service not ready, incompatible embeddings.	AI /models, FaceProfile, AI notes.
Attendance percentage wrong	Cancelled class counted wrongly, duplicate session, absent/present statuses misunderstood.	AttendanceRecord and attendanceAnalyticsService.js.
Exam eligibility strange	Schedule missing or exam date too near.	Classroom scheduleSlots and ClassExam.
Admin delete missing related data	Cascade rules need update.	adminService.js delete summaries.

Recommended tests

- Signup student, verify OTP, login.
- Signup teacher, verify OTP, login.
- Request password reset OTP, verify OTP, enter new password twice, login with new password.
- Create class, student joins by code.
- Enroll face with good images and reject bad image cases.
- Run manual attendance and check student dashboard percentage.
- Run QR attendance with valid and expired token cases.
- Run AI attendance and verify draft correction before final submission.
- Submit leave proof, teacher approves/rejects, student receives status.
- Set exam rule and check eligibility calculations.
- Archive class and confirm attendance actions are blocked.
- Admin deletes test class/user and verifies linked records are removed.

Chapter 27: Troubleshooting Cookbook

Use this chapter when something breaks. Each row gives a symptom, likely cause, and first places to inspect.

Symptom	First investigation
Signup OTP not arriving	Check <code>/api/v1/email/status</code> . Confirm <code>EMAIL_ENABLED</code> , <code>EMAIL_HOST</code> , <code>EMAIL_PORT</code> , <code>EMAIL_SECURE</code> , <code>EMAIL_USER</code> , and <code>EMAIL_PASSWORD</code> are set. Check spam folder. Check <code>EmailDeliveryLog</code> status. Do not print the password.
User sees verify email error on login	Open the User document and check <code>emailVerified</code> . If false, verify OTP or use admin email verification toggle during development.
Forgot password resend button disabled	This is expected for 60 seconds. Backend also enforces cooldown, so refreshing the browser should not bypass it.
Dashboard shows no classes	For students, check <code>ClassMembership</code> . For teachers, check <code>Classroom.teacherUserId</code> . Make sure <code>userId</code> is uppercase-normalized.
Join code not working	Check <code>Classroom.joinCode</code> , class status, and whether the student already has a membership.
AI says missing face profiles	The student has not completed <code>FaceProfile/enrollment</code> or AI local enrollment store does not contain the profile.
AI says re-enrollment required	The stored embedding model or dimensions do not match current ArcFace model settings.
Photo attendance detects no faces	Use clearer image, better lighting, less blur, front-facing students, and make sure AI service <code>/ready</code> is true.
Wrong student suggested present	Check confidence and margin. Use draft correction before finalization. Consider improving enrollment photos.
Attendance percentage too low	Check total counted classes and whether absences were final. Remember cancelled classes should not count.
Exam eligibility says at risk	Check required percentage, present count, total count, and remaining scheduled classes before exam date.
Teacher cannot take attendance	Class may be archived. Active operations are intentionally blocked for archived classes.
Admin delete feels incomplete	Check <code>adminService</code> deletion summary and related collections. Add cleanup for any new collection introduced later.
Email timeout appears	SMTP server did not answer before timeout. Network, firewall, Gmail, or port can be the cause.
Frontend route opens landing page unexpectedly	Check <code>App.jsx</code> route string and URL hash. Hash routes must use <code>##route-name</code> .

How to read backend errors

Backend services usually throw Error objects with user-readable messages. Controllers catch them and return JSON. If the frontend shows a message, search that exact message in `backend/src` to find where it was thrown. This is one of the fastest debugging tricks.

How to read frontend errors

React development errors often show the component stack and file line. Start at the topmost project file, not inside React internals. If the error says duplicate key, find the `.map` call rendering a list and check the key prop. If the error says cannot read property of undefined, inspect the data shape and loading state.

How to debug API calls

- Open browser developer tools and the Network tab.
- Click the action again.
- Check the request URL, method, payload, and status code.
- Read the JSON response message.

- Find the backend route for that URL in routes/index.js.
- Open the controller and service called by that route.

How to debug data

When a dashboard looks wrong, identify which model should contain the data. For class list, check ClassMembership and Classroom. For attendance percentage, check AttendanceRecord. For exam eligibility, check ClassExam and Classroom scheduleSlots. For leave proof, check LeaveRequest.

Chapter 28: Security, Limitations, And Future Improvements

Current security strengths

- Passwords are hashed with bcrypt.
- Email verification is required before login where configured.
- OTP records expire and are consumed after successful verification.
- OTP resend cooldown exists on both frontend and backend.
- Teacher class actions check class ownership.
- Admin destructive actions are separated from normal dashboards.
- Email status avoids exposing SMTP password.

Current limitations

- Frontend session storage is simple and not as strong as production secure cookie authentication.
- AI local JSON stores should be moved to durable database/object storage for production.
- Attachments stored as data URLs are simple but can grow database size; object storage is better long term.
- Face recognition needs consent, privacy policy, deletion flow, and access audit in real deployment.
- Role-based middleware could be stronger and centralized.
- Automated test coverage should be expanded across backend services and frontend flows.

Future roadmap

Priority	Improvement	Why
High	JWT or secure cookie authentication	Stronger production login sessions.
High	Central role middleware	Avoid repeating authorization checks.
High	Object storage for attachments/images	Prevents MongoDB from storing very large base64 data.
Medium	OpenAPI contract generation	Keeps frontend/backend payloads synchronized.
Medium	Background worker for emails and auto-finalization	More scalable than in-process jobs.
Medium	Audit logs for attendance edits	Shows who changed what and when.
Medium	AI confidence calibration dashboard	Helps tune thresholds using real data.
Low	Parent/guardian portal if required by institute	Optional extension, not necessary for current flow.

Chapter 29: Code Reading Workbook

This workbook is for studying the project. It gives small reading tasks that build confidence.

Task 1: trace login

- Open frontend/src/pages/AuthPage/AuthPage.jsx.
- Find handleLoginSubmit.
- See it call loginUser from services/api.js.
- Open backend/src/routes/index.js and find /auth/login.
- Open authController.js and find login.
- Open authService.js and find loginUser.
- Find the bcrypt password comparison.
- Return to AuthPage and see how route changes after login.

Task 2: trace student dashboard

- Open StudentDashboardPage.jsx and find the fetch function.
- Find fetchStudentDashboard in api.js.
- Find GET /students/:userId/dashboard in routes/index.js.
- Open studentController.js.
- Open studentDashboardService.js.
- Find where ClassMembership, AttendanceRecord, ClassExam, LeaveRequest, and FaceProfile are combined.

Task 3: trace AI attendance

- Open TeacherClassroomPage.jsx and find handleRunAttendance.
- Find processTeacherAttendance in api.js.
- Find the backend route /attendance/session.
- Open teacherClassroomController.js and teacherClassroomService.js.
- Find processClassroomAttendanceWithAi in aiService.js.
- Open ai-service/app/api/routes.py and find /attendance/classroom-recognition.
- Open classroom_attendance_pipeline.py and read process().

Task 4: trace exam eligibility

- Open TeacherExamsSection.jsx to see teacher input.
- Find setClassExam in api.js and backend routes.
- Open examScheduleService.js.
- Find buildStudentExamEligibility.
- Manually calculate one example and compare it with the code.

Task 5: trace email

- Open emailNotificationService.js.
- Find the event you care about, such as sendPasswordResetOtp or notifyAbsentStudents.
- Open emailTemplateService.js and find the matching template.
- Open emailService.js and find sendEmail.
- Find where EmailDeliveryLog is created.

Chapter 30: Example API Requests And Responses

These examples use dummy data. They show the shape of requests and responses so you can understand how frontend and backend communicate.

Signup request

```
POST /api/v1/auth/signup
Content-Type: application/json
```

```
{
  "role": "student",
  "firstName": "Asha",
  "lastName": "Kumar",
  "userId": "STU-1001",
  "rollNumber": "CSE-041",
  "password": "strongpass123",
  "email": "student@example.com",
  "department": "Computer Science"
}
```

Signup response

```
{
  "message": "Account created. Verify your email OTP before logging in.",
  "user": {
    "role": "student",
    "firstName": "Asha",
    "userId": "STU-1001",
    "emailVerified": false
  },
  "verification": {
    "expiresAt": "2026-05-20T10:20:00.000Z",
    "emailStatus": { "status": "sent" }
  }
}
```

Verify password reset OTP

```
POST /api/v1/auth/password-reset/verify
Content-Type: application/json
```

```
{
  "role": "student",
  "userId": "STU-1001",
  "otp": "123456"
}
```

Response:

```
{
  "message": "OTP verified. Enter your new password.",
  "resetSession": {
    "resetToken": "<temporary-reset-token>",
    "expiresAt": "2026-05-20T10:30:00.000Z"
  }
}
```

Create class

```
POST /api/v1/teachers/TCH-1001/classes
Content-Type: application/json
```

```
{
  "subjectName": "Artificial Intelligence",
  "subjectCode": "CSE-AI-301",
  "section": "A",
  "room": "Lab 2",
  "scheduleSlots": [
    { "day": "mon", "startTime": "10:00", "endTime": "11:00" },
    { "day": "wed", "startTime": "10:00", "endTime": "11:00" }
  ]
}
```


Run AI attendance

```
POST /api/v1/teachers/TCH-1001/classes/66500000000000000000101/attendance/session
Content-Type: application/json
```

```
{
  "captureImages": [
    "data:image/jpeg;base64,<omitted>"
  ]
}
```

Response includes:

- recognizedStudents
- reviewQueue
- unknownDetections
- absentStudents
- todayAttendanceDraft
- absenteeNotificationSummary

Set exam

```
PATCH /api/v1/teachers/TCH-1001/classes/66500000000000000000101/exam
Content-Type: application/json
```

```
{
  "title": "Mid Semester Exam",
  "examDate": "2026-06-10",
  "requiredAttendancePercentage": 75,
  "note": "Students below 75 percent must improve attendance."
}
```

Chapter 31: Complete Day In The Life Walkthrough

This chapter follows a realistic day from morning setup to final attendance analytics. It connects many features into one story.

Morning: teacher checks dashboard

The teacher logs in and opens the teacher dashboard. The frontend calls `GET /teachers/:userId/dashboard`. The backend loads the teacher, managed classes, memberships, attendance summaries, and upcoming exams. The dashboard shows today's schedule and which classes need attention.

Before class: students join and enroll

New students enter the join code. This creates `ClassMembership` documents. Students who have not enrolled face profiles open the face enrollment page and upload multiple reference images. The AI service builds embeddings and the backend saves `FaceProfile` metadata.

During class: teacher takes photo

The teacher opens the class workspace and takes a classroom photo. The frontend uses the camera, converts the image into a payload, and calls the backend. The backend checks class ownership and active status before forwarding to AI.

AI processing

The AI service detects visible faces and compares them against the enrolled profiles of only that class roster. It returns recognized students, low-confidence review items, unknown detections, and absent students. This response is not final attendance.

Draft creation and early email

The backend creates an `AttendanceDraft`. Students marked absent receive email. This email is intentionally early so a present student can report the mistake before final records are saved.

Teacher correction

The teacher checks the draft. Maybe one student was hidden behind another student and got marked absent. The teacher changes that student to present and saves the draft. The source can show teacher-edited so later readers know AI was corrected.

Final submission

The teacher clicks final submission. The backend converts the draft into `AttendanceRecord` documents. Dashboards now use those final records to update percentages, flagged students, trends, and exam eligibility.

After class: student checks dashboard

A student opens the dashboard and sees that attendance changed. If an upcoming exam exists, the eligibility card recalculates how many future classes must be attended. If attendance is below requirement, alerts guide the student.

End of semester

When the subject is finished, the teacher archives the class. The summary remains visible but new attendance actions are blocked. This keeps historical data clean.

Chapter 32: Viva And Presentation Question Bank

Use these questions for practice before explaining the project to a teacher, examiner, or teammate.

Question	Short answer
What problem does SAMS solve?	It records attendance and makes it useful through analytics, alerts, exam eligibility, leave proof, and teacher-reviewed AI attendance.
Why are there three services?	React handles UI, Express handles business rules and database, and FastAPI handles AI inference. This separation keeps responsibilities clean.
Why does the frontend not call AI directly?	The backend must validate teacher, class, roster, and finalization rules before and after AI inference.
Why use MongoDB?	It stores flexible documents such as schedules, drafts, attachments, email metadata, AI notes, and dashboard-related records.
What is a Mongoose model?	A schema-backed JavaScript model used by the backend to read and write MongoDB documents.
Why hash passwords?	So the original password is not stored. Login compares the submitted password with the stored hash using bcrypt.
Why is email verification required?	It proves the user owns the email address before the account becomes active.
Why is OTP hashed?	If the database leaks, active OTP codes should not be directly readable.
Why add OTP resend cooldown?	To reduce spam, abuse, and accidental repeated mail requests.
What is Today Attendance Data?	An editable AttendanceDraft created before final attendance records are saved.
Why not directly save AI attendance?	AI can make mistakes. Teacher correction protects students from wrong final records.
Why send absent email before finalization?	It allows students to report mistakes early while the teacher can still correct the draft.
What is ArcFace?	A face recognition method that creates embeddings useful for comparing whether two face images belong to the same person.
What is an embedding?	A vector of numbers that represents identity-related face features.
Why use multiple enrollment images?	To build a more reliable average embedding and reject poor images.
What happens if a student is not enrolled?	AI may list them as missing profile or absent; teacher can still mark manually.
What is recognition threshold?	A confidence cutoff for accepting a face match automatically.
What is recognition margin?	A gap between best and second-best matches to reduce confused identity matches.
Why can a class be archived?	To preserve history after the semester while blocking new attendance actions.
Why is cancelled class separate from absent?	Cancelled means class did not happen, so students should not lose attendance percentage.
How is attendance percentage calculated?	Present or late count divided by total counted classes, multiplied by 100.
How is exam eligibility calculated?	The system counts remaining scheduled classes before exam and calculates minimum classes needed to reach required percentage.
What is ClassMembership?	A link document connecting one student to one classroom.
What is the role of EmailDeliveryLog?	It records sent, skipped, or failed email attempts for debugging and audit.
What is the admin panel for?	System record cleanup and account/class control, not normal teaching workflows.

Long answer structure

For any feature question, answer in this order: purpose, user action, frontend file, backend route, service logic, database model, edge cases, and improvement. This structure makes your explanation sound organized.

Example long answer: AI attendance

Purpose: reduce manual effort while keeping teacher control. User action: teacher captures classroom photo. Frontend file: TeacherClassroomPage handles image capture and API call. Backend route: /teachers/:teacherId/classes/:classId/attendance/session. Service logic: teacherClassroomService validates ownership and active class, builds roster payload, calls aiService, creates AttendanceDraft, sends absent emails. AI logic: FastAPI detects faces, builds embeddings, matches roster profiles using ArcFace and thresholds. Database: AttendanceDraft first, AttendanceRecord after finalization. Edge case: missing enrollment or low confidence requires manual review. Improvement: add audit log and model confidence calibration dashboard.

Chapter 33: Appendices

Quick command reference

```
# Install Node workspaces
npm install

# Run all services
npm run dev

# Run frontend only
npm run dev:frontend

# Run backend only
npm run dev:backend

# Run AI service
cd ai-service
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000

# Frontend production build
npm --workspace frontend run build

# Backend health
curl http://localhost:4000/api/v1/health
```

Glossary

Term	Meaning
API	A set of URLs the frontend can call to ask the backend for work.
Backend	The server-side code that validates requests, talks to the database, sends email, and calls AI.
Controller	A backend function that receives an HTTP request and returns an HTTP response.
Service	Backend business logic separated from controllers so it can be reused and tested.
Model	A database schema definition that says what fields a document can have.
Mongoose	A library that lets Node.js work with MongoDB using schemas and models.
Document	One stored MongoDB object, similar to one row in a SQL database.
Collection	A group of MongoDB documents, similar to a table.
OTP	One time password. A temporary code sent to email to prove account ownership.
SMTP	The email protocol used by the backend to send real emails.
Hash	A one-way representation of a password or OTP. The original value is not stored.
Embedding	A vector of numbers representing a face in a way a model can compare.
Cosine similarity	A score that measures how close two embeddings point in vector space.
Threshold	A chosen cut-off value. If confidence is above it, the system accepts a match.
Draft	Editable temporary attendance data before final records are saved.
Finalization	The step that writes permanent attendance records.
Roster	The list of students belonging to one class.
Join code	A teacher-generated code students use to join a class.
CORS	Browser security rule controlling which frontend origins can call the backend.
Environment variable	A runtime setting stored outside code, such as database URL or SMTP host.

Beginner learning checklist

- Explain the four services without looking at the diagram.
- Trace signup from frontend form to User and EmailOtp models.
- Trace teacher class creation to Classroom model.
- Trace student joining class to ClassMembership model.
- Calculate attendance percentage manually from sample records.
- Explain why AI creates suggestions, not final records.
- Explain what an embedding is in one sentence.
- Explain why OTP must expire and be hashed.
- Explain why archive is safer than delete for old classes.
- Open backend routes and identify controller and service for one endpoint.

Files to read after this guide

File	Why read it
frontend/src/App.jsx	Understand every page route.
frontend/src/services/api.js	See all frontend-backend calls.
backend/src/routes/index.js	See every backend endpoint.
backend/src/services/authService.js	Understand signup, login, OTP, password reset.
backend/src/services/teacherClassroomService.js	Understand attendance and class workspace logic.
backend/src/services/studentDashboardService.js	Understand how student dashboard data is built.
backend/src/services/attendanceAnalyticsService.js	Understand attendance percentage calculations.
backend/src/services/examScheduleService.js	Understand exam eligibility math.
ai-service/app/pipelines/classroom_attendance_pipeline.py	Understand AI attendance flow.
ai-service/app/services/face_recognition_service.py	Understand embeddings, thresholds, and model compatibility.